ALCOR academy

TRAINING PROGRAMME



Lesson 2

Test Doubles

“Functions that change state should not return values and functions that return values should not change state.”

Bertrand Meyer, Object-Oriented Software Construction

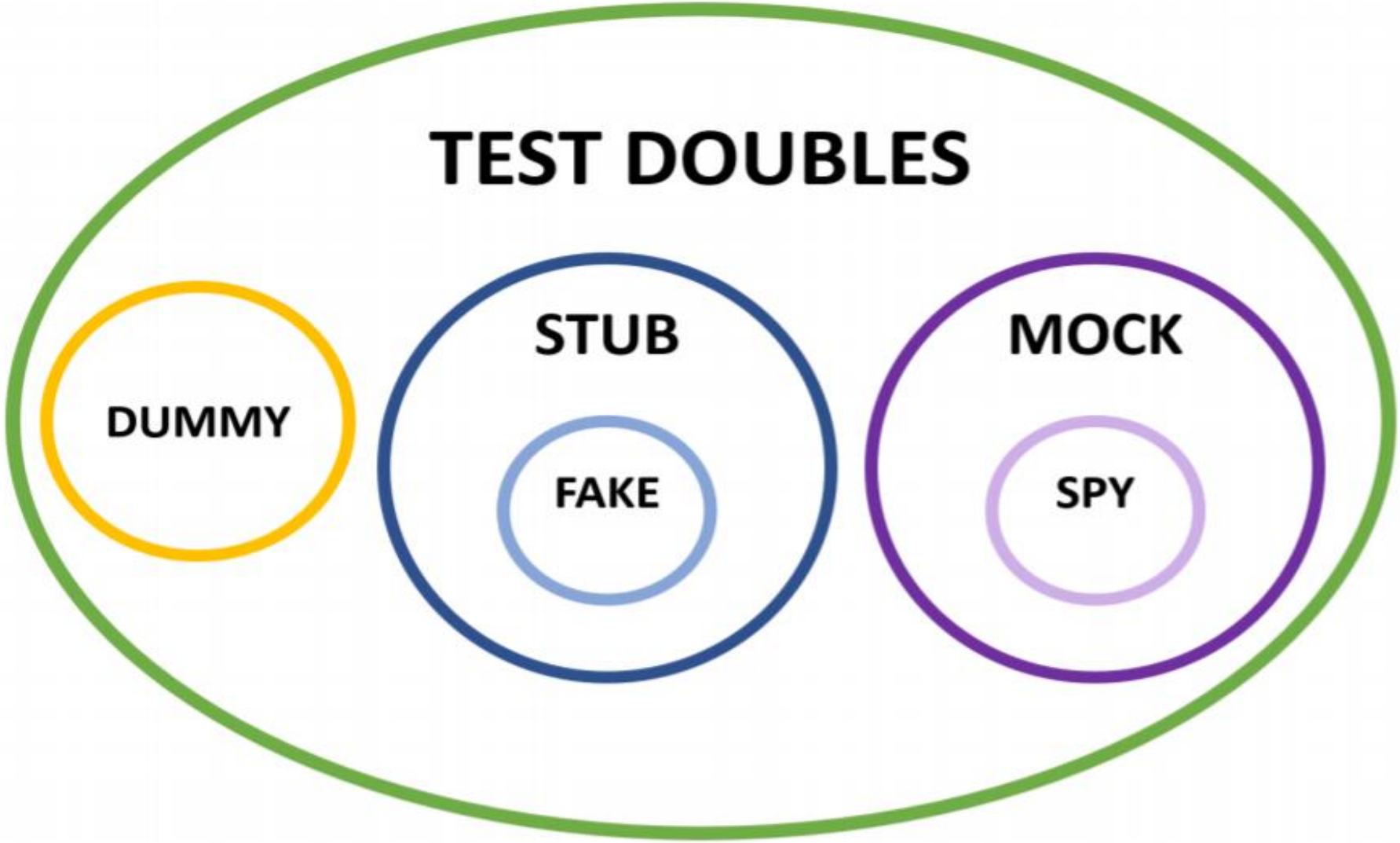
Test Doubles Taxonomy

→ **Dummy**: needed to complete the parameters' list of a method, but never used. Not common in well-designed systems.

→ **Stub**: responds to calls with some pre-programmed output. They need to be specifically setup for every test. **Fakes** are hand made stubs.

→ **Mock**: set up with expectations of the calls they are expected to receive. Provide a way of verify that a behaviour has been triggered correctly. **Spy** is a hand made mock.

Test Doubles Taxonomy



ALCOR academy

Command-Query Separation

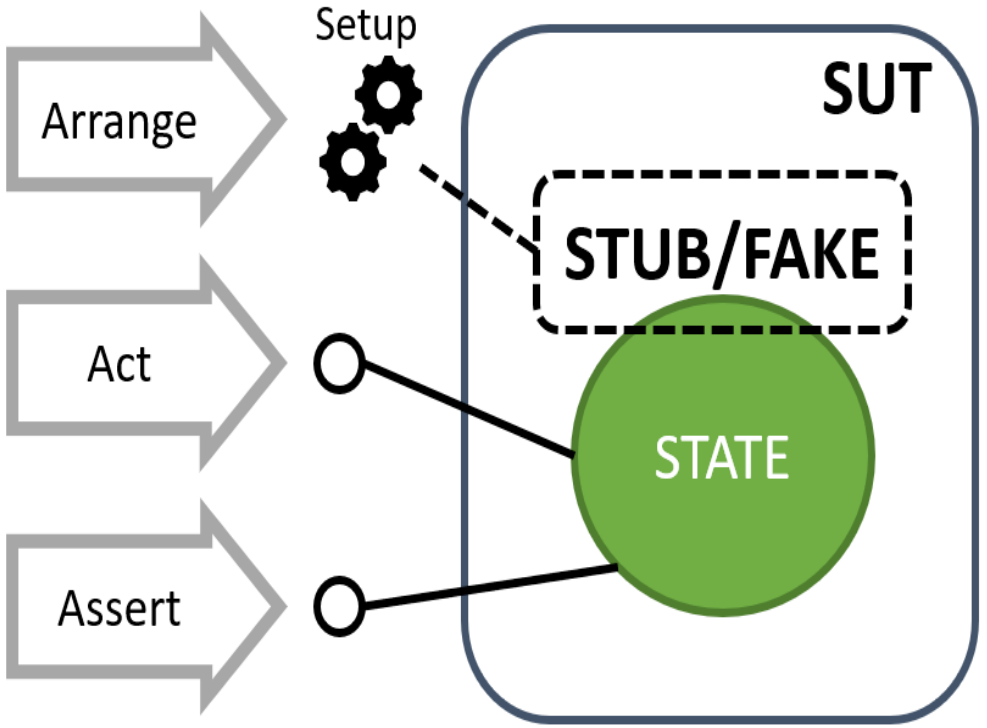
There are essentially only two categories of abstractions designed with two types of methods:

- **Command** methods modify or mutate the state underneath the interface, but does not answer any portion of that state.
- **Query** methods answer the current state beneath the interface, without modify that state.

Use stubs for queries

A **query** is a method call that returns data and shouldn't have any effect on the state of an object.

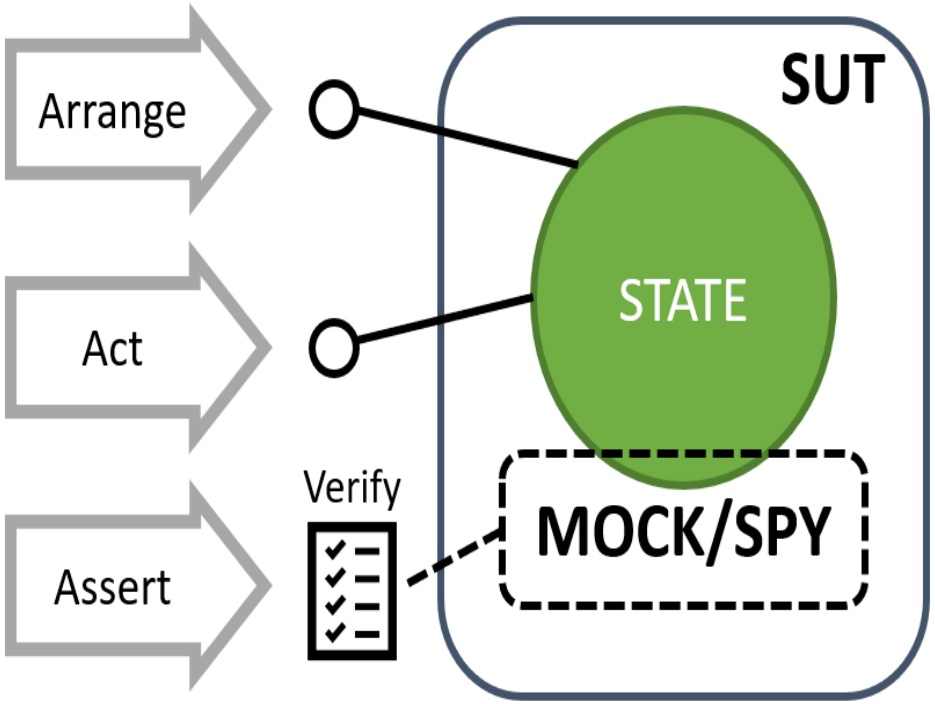
Stubs respond to calls made during the test with some pre-programmed output



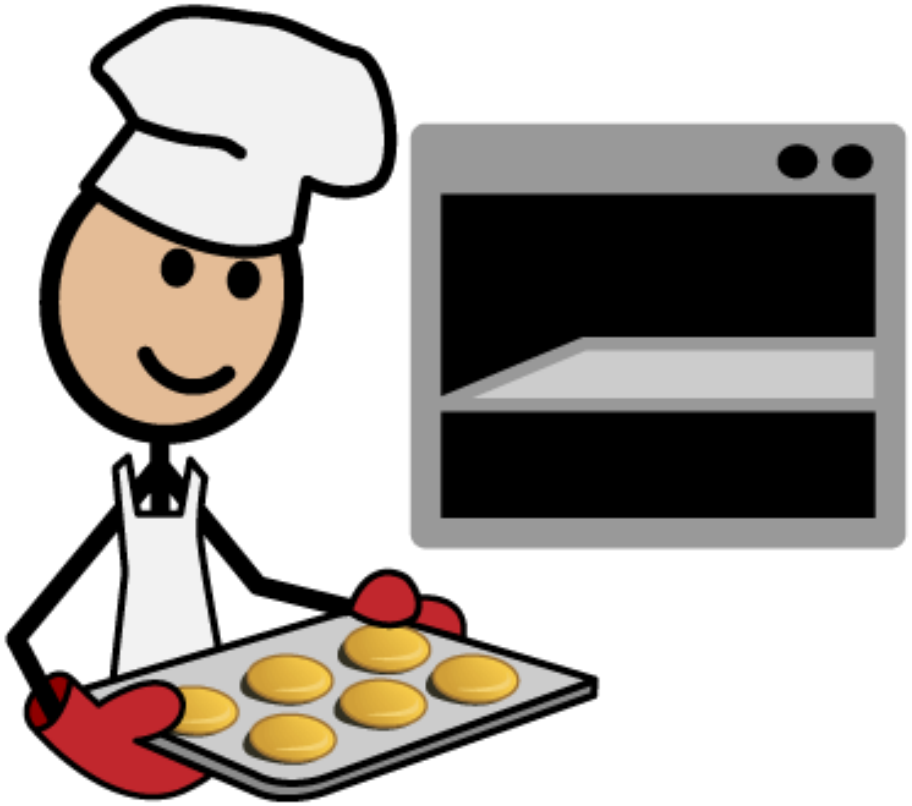
Use mocks for commands

A **command** is a method call that changes the state of the system.

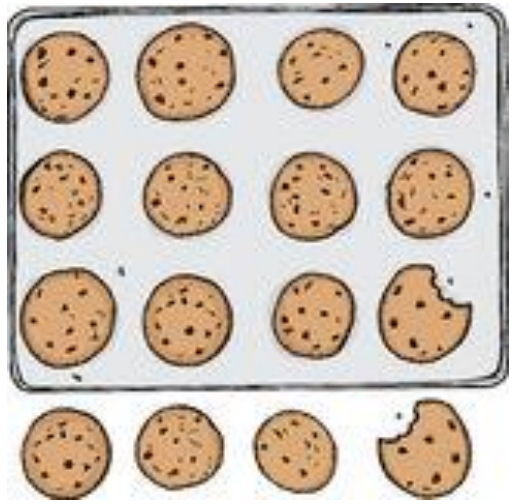
Mocks are used to provide a way of confirming that commands have been triggered correctly



Bake

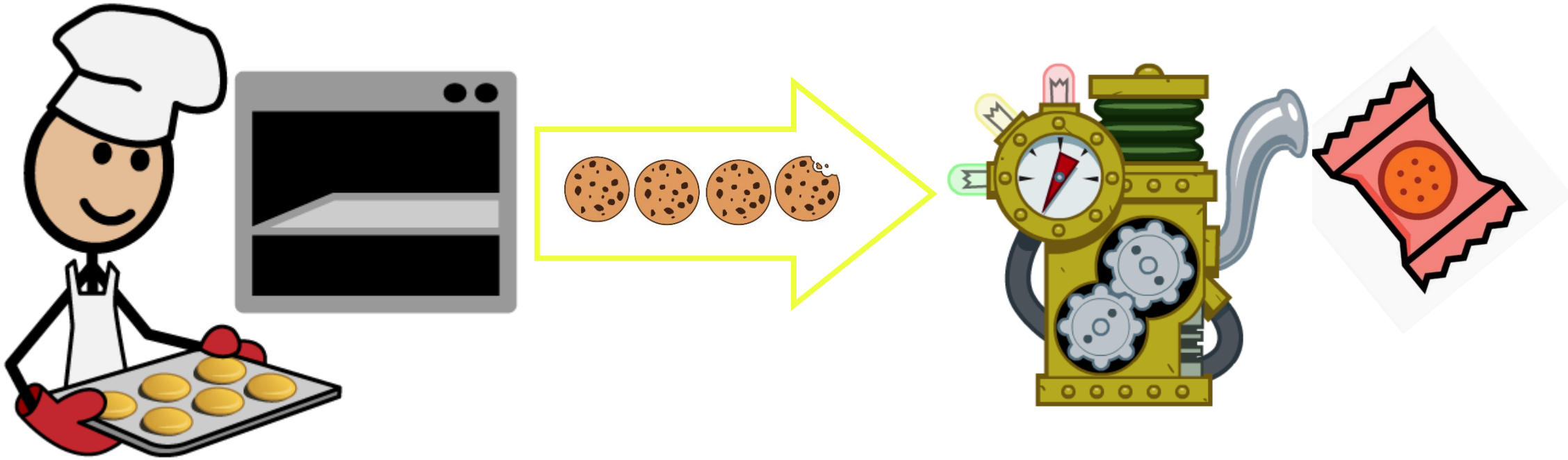


Assert



Bake

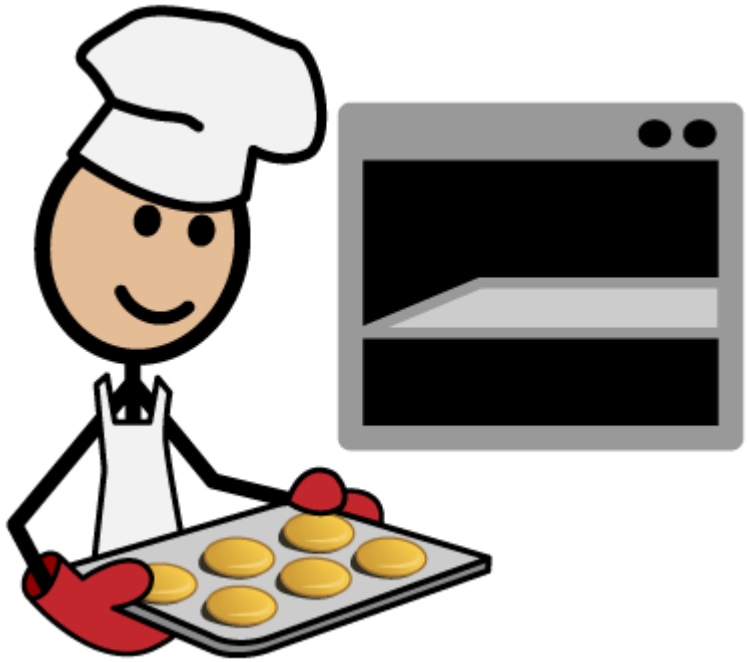
Package

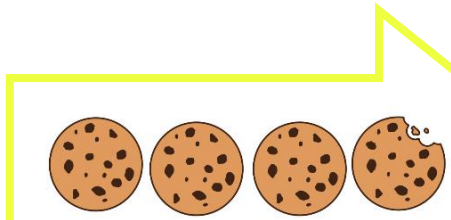


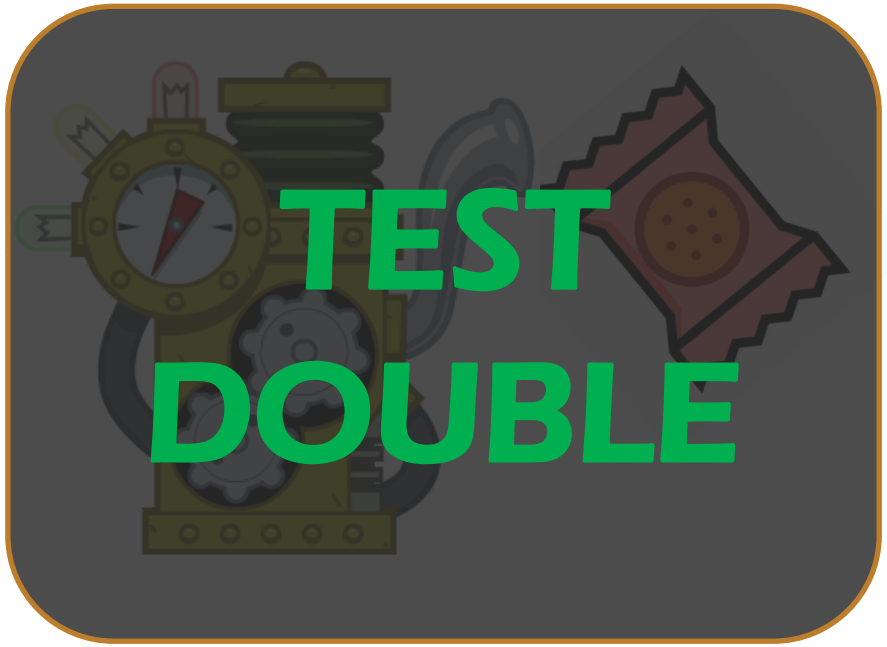
Assert ?

Bake

Package







Assert

Test Doubles Guidelines

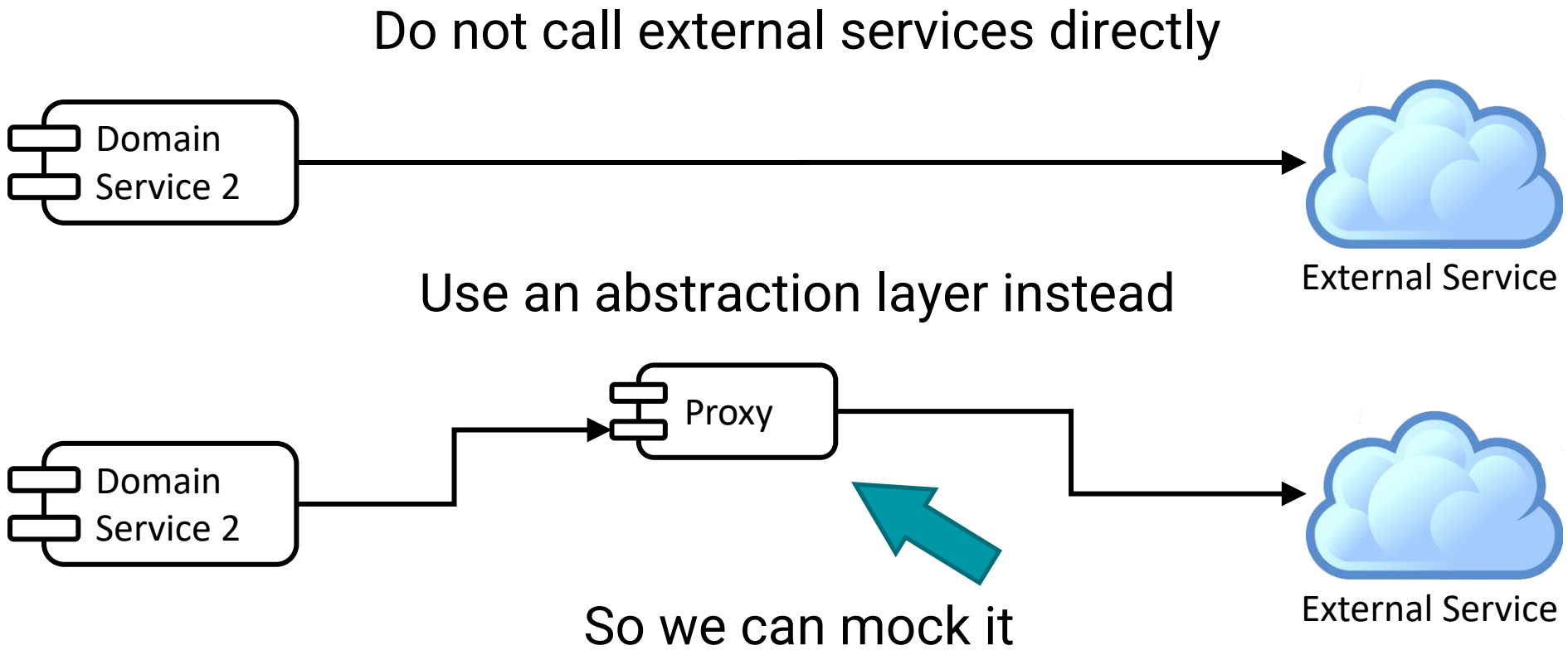
→ Only use Test Doubles for classes that you own

If the Test Doubles are coupled with an external interface, they cannot change the design to respond to requirements. Using a thin abstraction layer separating the external call from our system (like a *proxy* or a *façade*) allows to:

1) Implement an interface functional to the design of our system
2) Substitute a class we own with a Test Double

Test Doubles Guidelines

→ Only use Test Doubles for classes that you own



Test Doubles Guidelines

→ Verify as little as possible in a test

“A specification should be as precise as possible, but not more precise.”

When tests have been over specified to check features that are implementation details instead of the expression of some requirement, they tend to become fragile and fail for unrelated changes.

Test Doubles Guidelines

→ Don’t use Test Doubles for isolated objects

If an object has no collaborations with other objects in the system, it does not need to be tested using Mock/Stub Objects.

Aggregates, entities and generally the domain model fall into this category and almost never need Test Doubles.

Test Doubles Guidelines

→ Don’t add behaviour inside Test Doubles

Mock objects might also be Stubs, but should not add any additional complexity to the test environment.

Their behaviour should be obvious and self explanatory.

We have noticed that when real behaviour has been added to Mock objects, it is usually a strong symptom of misplaced responsibilities.

Test Doubles Guidelines

→ **Only use Test Doubles for immediate neighbours**

→ **Too many Test Doubles**

An object that has to navigate a tree of objects for completing its task is likely to be fragile because it has too many dependencies crossing several abstraction layers. High probability of improving the design looking for code smells like Feature Envy, Inappropriate Intimacy, Message Chain and act accordingly.

Often caused by the violation of the Law of Demeter.



Test Doubles Guidelines



ATTENTION

The same class CAN act BOTH

as a Stub AND as a Mock

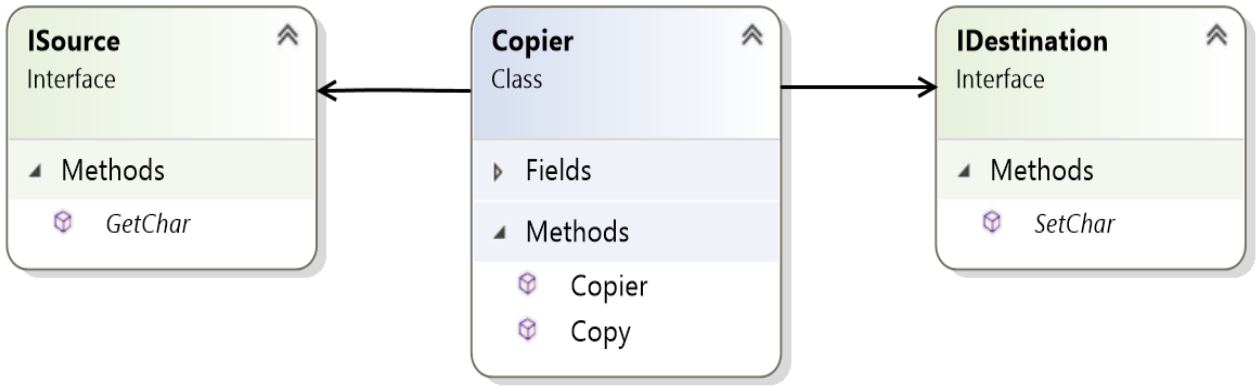
The concept of TEST DOUBLE is resolved AT A METHOD LEVEL.

That means that depending on the method we are considering it might be a Stub or a Mock.



KATA – Character Copier

The character copier is a simple class that reads characters from a source and copies them to a destination one character at a time.
When the method Copy is called on the copier then it should read characters from the source and copy them to the destination until the source returns a newline (\n).
The exercise is to implement the character copier using Test Doubles for the source and the destination (try using Spies – manually written Mocks – and Mocks written with a mocking framework). Start from these definitions:




```
public class Copier
{
  public Copier(ISource source, IDestination destination)
  {...}
  public void Copy() {}
}

public interface ISource
{
  char GetChar();
}

public interface IDestination
{
  void SetChar(char character);
}
```



KATA – Instrument Processor

In the Instrument Processor kata, we are going to implement a class that gets tasks from a `TaskDispatcher` and executes them on an `Instrument`.
The `InstrumentProcessor` must implement the following interface:

```
public interface IInstrumentProcessor
{
  void Process();
}
```

The task dispatcher must have the following interface:

```
public interface ITaskDispatcher
{
  string GetTask();
  void FinishedTask(string task);
}
```

The `GetTask` method returns the next task to execute on the instrument. After the task was successfully executed on the instrument, the `FinishedTask` method must be called by the `InstrumentProcessor` passing the task that was completed as the method argument.



KATA – Instrument Processor

The InstrumentProcessor has the following interface:

```
public interface IInstrument
{
  void Execute(string task);
  event EventHandler Finished;
  event EventHandler Error;
}
```

The `Execute` method starts the instrument, which will begin to execute the task passed to it. The method will return immediately (we assume that the instrument implementation is asynchronous).

The `Execute` method will throw an `ArgumentNullException` if the task passed in is null. When the instrument finishes executing then the `Finished` event is fired.

When the instrument detects an error situation during executing (note that the Execute method will already have returned the control flow to the caller due to its asynchronous implementation) then it fires the Error event.



KATA – Instrument Processor

The exercise is to implement the `InstrumentProcessor` in a way that:

- When the method `Process` is called then the `InstrumentProcessor` gets the next task from the task dispatcher and executes it on the instrument.
- When the `Execute` method of the instrument throws an exception then this exception is passed on to the caller of the `Process` method.
- When the instrument fires the finished event then the `InstrumentProcessor` calls the task dispatcher's `FinishedTask` method with the correct task.
- When the instrument fires the `Error` event then the `InstrumentProcessor` writes the string "Error occurred" to the console.

